LATIHAN TERSTRUKTUR
PERTEMUAN 4
IMPLEMENTASI UNIT TESTING DENGAN GITHUB ACTIONS

Git & GitHub, JUnit, GitHub Actions & Code Coverage



Disusun oleh:

Nama	Umul Banin
NIM	1224023
Kelas	Reguler Pagi
Tanggal	10 Oktober 2026
Mata Kuliah	Implementasi Pengujian Perangkat Lunak

STMIK Bandung

BAGIAN A: GIT & GITHUB DASAR

Soal 1

1. Perbedaan Centralized VCS dan Distributed VCS

Pada **Centralized VCS (CVCS)**, seluruh versi file disimpan di satu server pusat dan developer bekerja langsung terhubung ke server tersebut. Akibatnya, kalau server mati maka semua orang tidak bisa commit atau mengambil versi lama (single point of failure), dan pekerjaan sulit dilakukan saat offline. Pada **Distributed VCS (DVCS)**, setiap developer memiliki salinan lengkap repository beserta seluruh history-nya di komputer masing-masing. Karena itu commit, melihat log, dan membuat branch bisa dilakukan secara offline, dan jika satu server rusak repository masih bisa dipulihkan dari salinan developer lain.

Aspek	**Centralized VCS**	**Distributed VCS**
Penyimpanan	Satu server pusat	Setiap developer punya salinan lengkap
Kerja offline	Tidak bisa	Bisa
Single point of failure	Ada	Tidak ada
Branching	Relatif berat	Cepat dan mudah
Contoh	SVN (Subversion), CVS	Git, Mercurial

2. Lima Git command yang wajib dikuasai

No	**Command**	**Fungsi**
1	`git init`	Membuat repository Git baru di folder lokal.
2	`git add`	Memasukkan file yang berubah ke staging area sebelum di-commit.
3	`git commit`	Menyimpan perubahan di staging area ke history repository beserta pesan commit.
4	`git push`	Mengirim commit dari repository lokal ke remote repository (GitHub).
5	`git pull`	Mengambil dan menggabungkan perubahan dari remote ke repository lokal.

Command pendukung lain yang juga sering dipakai: `git branch`, `git checkout`, `git merge`, dan `git log`.

3. Fungsi .gitignore

File `.gitignore` berisi daftar file atau folder yang tidak ingin dilacak dan tidak boleh ikut ter-commit ke repository. Biasanya isinya file hasil build (misalnya `target/`), file konfigurasi IDE (`.idea/`, `.vscode/`), file sistem operasi (`.DS_Store`), file sementara seperti `__pycache__/`, serta data sensitif seperti password atau API key. Dengan begitu repository tetap bersih, ukurannya kecil, dan tidak terjadi konflik akibat file yang dihasilkan otomatis oleh masing-masing komputer.

Soal 2

Berikut command yang saya gunakan untuk setiap langkah (username GitHub saya: umulbaninn).

```
# 1. Buat repository di GitHub (lewat web)
#    Nama      : latihan-git-1224023
#    Visibility: Public (tanpa centang "Add a README file")

# 2. Clone repository ke lokal
git clone https://github.com/umulbaninn/latihan-git-1224023.git
cd latihan-git-1224023

# 3. Buat README.md berisi nama dan NIM
echo Nama : Umul Banin> README.md
echo NIM : 1224023>> README.md

# 4. Commit file README.md
git add README.md
git commit -m "docs: add README"

# 5. Push ke GitHub
git branch -M main
git push -u origin main

# 6. Buat branch baru bernama feature-test
git branch feature-test

# 7. Pindah ke branch tersebut
git checkout feature-test

# 8. Buat test.txt lalu commit
echo ini file untuk testing> test.txt
git add test.txt
git commit -m "test: add test file"

# 9. Push branch feature-test ke GitHub
git push -u origin feature-test

# 10. Merge feature-test ke main
git checkout main
git merge feature-test
git push origin main
```

Catatan: pada langkah 5, perintah `git branch -M main` dipakai untuk memastikan nama branch lokal adalah *main* (sebagian versi Git masih membuat branch awal bernama *master*). Perintah `git push origin main` di akhir ditambahkan supaya hasil merge juga masuk ke GitHub.

BAGIAN B: UNIT TESTING FRAMEWORK

Soal 3

1. Test case berdasarkan independent path (5 poin)

Untuk method `validasiPendaftaran()` saya memakai hasil analisis di Tugas Minggu 3 (Gambar 3): N = 11, E = 15, dan predicate ada 5 (P1 sampai P5). Maka CC = 15 - 11 + 2 = 6, sama dengan 5 + 1 = 6, sehingga ada **6 independent path**. Jumlah path-nya lebih banyak satu dari jumlah `if` karena kondisi email dipecah menjadi dua predicate (P3: `email == null` dan P4: `!email.contains("@")`).

Pada soal tertulis 5 test case, tetapi independent path hasil Minggu 3 ada 6. Supaya konsisten dengan Minggu 3, saya buat **6 test case**, satu untuk setiap path.

Path	Jalur	Kondisi
1	1 -> 2 -> E	P1 Yes: nama null atau kosong
2	1 -> 3 -> 4 -> E	P1 No, P2 Yes: umur < 17
3	1 -> 3 -> 5 -> 7 -> E	P1 No, P2 No, P3 Yes: email null
4	1 -> 3 -> 5 -> 6 -> 7 -> E	P3 No, P4 Yes: email tidak ada "@"
5	1 -> 3 -> 5 -> 6 -> 8 -> 9 -> E	P4 No, P5 Yes: setujuSyarat = false
6	1 -> 3 -> 5 -> 6 -> 8 -> 10 -> E	P5 No: semua input valid

Path	nama	umur	email	setujuSyarat	Expected Output
1	""	20	budi@mail.com	true	Nama harus diisi
2	Budi	16	budi@mail.com	true	Umur minimal 17 tahun
3	Budi	20	null	true	Email tidak valid
4	Budi	20	budimail.com	true	Email tidak valid
5	Budi	20	budi@mail.com	false	Harus menyetujui syarat dan ketentuan
6	Budi	20	budi@mail.com	true	Pendaftaran berhasil

Seperti di Minggu 3, untuk Path 1 saya tambahkan satu test untuk nama = null (karena P1 punya dua kondisi: null dan kosong), dan satu test tambahan untuk nilai batas umur = 17.

2. Implementasi unit test JUnit 5

Kode yang diuji (saya letakkan dalam class `PendaftaranService`):

```
package com.stmik;

public class PendaftaranService {

    public String validasiPendaftaran(String nama, int umur, String email,
                                      boolean setujuSyarat) {
        if (nama == null || nama.isEmpty()) {            // P1
            return "Nama harus diisi";
        }
        if (umur < 17) {                                 // P2
            return "Umur minimal 17 tahun";
        }
        if (email == null || !email.contains("@")) {     // P3, P4
            return "Email tidak valid";
        }
        if (!setujuSyarat) {                             // P5
            return "Harus menyetujui syarat dan ketentuan";
        }
        return "Pendaftaran berhasil";
    }
}
```

Kode unit test (pola Arrange-Act-Assert):

```
package com.stmik;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class PendaftaranServiceTest {

    private PendaftaranService service;

    @BeforeEach
    public void setUp() {
        service = new PendaftaranService();
    }

    @Test
    public void testPath1_NamaKosong() {
        // Arrange
        String nama = "";
        // Act
        String hasil = service.validasiPendaftaran(nama, 20, "budi@mail.com", true);
        // Assert
        assertEquals("Nama harus diisi", hasil);
    }

    @Test
    public void testPath2_UmurKurangDari17() {
        String hasil = service.validasiPendaftaran("Budi", 16, "budi@mail.com", true);
        assertEquals("Umur minimal 17 tahun", hasil);
    }

    @Test
    public void testPath3_EmailNull() {
        String hasil = service.validasiPendaftaran("Budi", 20, null, true);
        assertEquals("Email tidak valid", hasil);
    }

    @Test
    public void testPath4_EmailTanpaAt() {
        String hasil = service.validasiPendaftaran("Budi", 20, "budimail.com", true);
        assertEquals("Email tidak valid", hasil);
    }

    @Test
    public void testPath5_SyaratTidakDisetujui() {
```

```
        String hasil = service.validasiPendaftaran("Budi", 20, "budi@mail.com", false);
        assertEquals("Harus menyetujui syarat dan ketentuan", hasil);
    }

    @Test
    public void testPath6_PendaftaranBerhasil() {
        String hasil = service.validasiPendaftaran("Budi", 20, "budi@mail.com", true);
        assertEquals("Pendaftaran berhasil", hasil);
    }

    // ---- test tambahan ----
    @Test
    public void testTambahan_NamaNull() {
        String hasil = service.validasiPendaftaran(null, 20, "budi@mail.com", true);
        assertEquals("Nama harus diisi", hasil);
    }

    @Test
    public void testTambahan_UmurBatas17() {
        String hasil = service.validasiPendaftaran("Budi", 17, "budi@mail.com", true);
        assertEquals("Pendaftaran berhasil", hasil);
    }
}
```

Soal 4

1. Cyclomatic Complexity calculateGrade()

Flow graph method `calculateGrade()` saya buat dengan penomoran node sebagai berikut:

Node	Isi	Node	Isi
1	if (score < 0 || score > 100)	7	if (score >= 60)
2	throw IllegalArgumentException	8	return "C"
3	if (score >= 80)	9	if (score >= 50)
4	return "A"	10	return "D"
5	if (score >= 70)	11	return "E"
6	return "B"	E	exit (akhir method)

Edge: 1-2, 1-3, 2-E, 3-4, 3-5, 4-E, 5-6, 5-7, 6-E, 7-8, 7-9, 8-E, 9-10, 9-11, 10-E, 11-E.

Rumus	Perhitungan	Hasil
V(G) = E - N + 2	16 - 12 + 2	6
V(G) = P + 1 (P = predicate node)	5 + 1 (node 1, 3, 5, 7, 9)	6
V(G) = jumlah region	5 region tertutup + 1 region luar	6

Cyclomatic Complexity = 6, artinya minimal dibutuhkan 6 test case. (Jika kondisi gabungan *score < 0 || score > 100* dihitung sebagai 2 predicate, nilainya menjadi 7. Di sini saya hitung per percabangan `if` sama seperti catatan di Tugas Minggu 3.)

2. Independent Path calculateGrade()

Path	Jalur node	Kondisi	Hasil
1	1-2-E	score < 0 atau score > 100	Exception
2	1-3-4-E	score >= 80	"A"
3	1-3-5-6-E	70 <= score < 80	"B"
4	1-3-5-7-8-E	60 <= score < 70	"C"
5	1-3-5-7-9-10-E	50 <= score < 60	"D"
6	1-3-5-7-9-11-E	0 <= score < 50	"E"

Contoh data uji tiap path: -1 (path 1), 85 (path 2), 75 (path 3), 65 (path 4), 55 (path 5), dan 40 (path 6).

3. Unit test JUnit untuk calculateGrade() dan isPassed()

Test mencakup 6 independent path, nilai batas (0, 49, 50, 59, 60, 69, 70, 79, 80, 100), serta pengecekan exception untuk nilai di luar rentang 0 sampai 100 pada kedua method.

```
package com.stmik;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.CsvSource;
import static org.junit.jupiter.api.Assertions.*;

public class StudentGradeTest {

    private StudentGrade studentGrade;

    @BeforeEach
    public void setUp() {
        studentGrade = new StudentGrade();
    }

    // ===== calculateGrade: 6 independent path =====
    @Test
    public void testCalculateGrade_Path1_ScoreNegatif() {
        IllegalArgumentException e = assertThrows(IllegalArgumentException.class,
                () -> studentGrade.calculateGrade(-1));
        assertEquals("Score must be between 0 and 100", e.getMessage());
    }

    @Test
    public void testCalculateGrade_Path1_ScoreLebihDari100() {
        assertThrows(IllegalArgumentException.class,
                () -> studentGrade.calculateGrade(101));
    }

    @Test
    public void testCalculateGrade_Path2_NilaiA() {
        assertEquals("A", studentGrade.calculateGrade(85));
    }

    @Test
    public void testCalculateGrade_Path3_NilaiB() {
        assertEquals("B", studentGrade.calculateGrade(75));
    }

    @Test
    public void testCalculateGrade_Path4_NilaiC() {
        assertEquals("C", studentGrade.calculateGrade(65));
    }

    @Test
    public void testCalculateGrade_Path5_NilaiD() {
        assertEquals("D", studentGrade.calculateGrade(55));
    }

    @Test
    public void testCalculateGrade_Path6_NilaiE() {
        assertEquals("E", studentGrade.calculateGrade(40));
    }

    // ===== calculateGrade: boundary value =====
    @ParameterizedTest
    @CsvSource({
        "0,E", "49,E", "50,D", "59,D", "60,C",
        "69,C", "70,B", "79,B", "80,A", "100,A"
    })
    public void testCalculateGrade_NilaiBatas(int score, String expected) {
        assertEquals(expected, studentGrade.calculateGrade(score));
    }
```

```
    // ===== isPassed =====
    @Test
    public void testIsPassed_Lulus() {
        assertTrue(studentGrade.isPassed(75));
    }

    @Test
    public void testIsPassed_BatasLulus60() {
        assertTrue(studentGrade.isPassed(60));
    }

    @Test
    public void testIsPassed_BatasTidakLulus59() {
        assertFalse(studentGrade.isPassed(59));
    }

    @Test
    public void testIsPassed_NilaiMinimum0() {
        assertFalse(studentGrade.isPassed(0));
    }

    @Test
    public void testIsPassed_NilaiMaksimum100() {
        assertTrue(studentGrade.isPassed(100));
    }

    @Test
    public void testIsPassed_ScoreNegatif() {
        assertThrows(IllegalArgumentException.class,
                () -> studentGrade.isPassed(-5));
    }

    @Test
    public void testIsPassed_ScoreLebihDari100() {
        assertThrows(IllegalArgumentException.class,
                () -> studentGrade.isPassed(101));
    }
}
```

Dependency yang dibutuhkan: `org.junit.jupiter:junit-jupiter:5.10.2` (sudah termasuk modul *junit-jupiter-params* untuk `@ParameterizedTest`).

BAGIAN C: GITHUB ACTIONS

Soal 5

1. Nama workflow

Nama workflow adalah **Python CI** (sesuai baris `name: Python CI`), yang akan tampil di tab Actions pada GitHub.

2. Trigger / event

- **push** ke branch `main` atau `develop`.
- **pull_request** yang menuju branch `main`.

3. Fungsi strategy matrix

Bagian `strategy: matrix: python-version: ['3.10', '3.11', '3.12']` membuat GitHub Actions menjalankan job `test` sebanyak tiga kali secara paralel, masing-masing dengan versi Python berbeda (3.10, 3.11, dan 3.12). Nilai versi diambil lewat variabel `${{ matrix.python-version }}` pada step Setup Python. Tujuannya untuk memastikan kode dan test tetap berjalan baik di beberapa versi Python (uji kompatibilitas) tanpa harus menulis tiga job terpisah, dan karena berjalan paralel waktunya juga lebih singkat.

4. Step pada job test

No	Nama step	Fungsi
1	Checkout code	Mengambil kode repository ke runner (actions/checkout@v4).
2	Setup Python ${{ matrix.python-version }}	Memasang Python sesuai versi pada matrix (actions/setup-python@v5).
3	Install dependencies	Upgrade pip lalu install pytest dan pytest-cov.
4	Run tests	Menjalankan `pytest --cov=src --cov-report=xml` untuk test sekaligus coverage.

Catatan: setelah saya cek, workflow pada soal ini hanya memiliki **4 step**, bukan 5. Jadi yang saya tuliskan adalah keempat step di atas. Step kelima ("Upload coverage report") hanya ada pada contoh workflow di slide materi, tidak ada pada workflow di soal.

Soal 6

File: `.github/workflows/test.yml`

```
name: Java Unit Test

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]

jobs:
  test:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Setup JDK 17
        uses: actions/setup-java@v4
        with:
          java-version: '17'
          distribution: 'temurin'

      - name: Run tests
        run: mvn test

      - name: Generate coverage report
        run: mvn jacoco:report

      - name: Upload artifact jacoco-report
        uses: actions/upload-artifact@v4
        with:
          name: jacoco-report
          path: target/site/jacoco/
```

Laporan JaCoCo dibuat di folder `target/site/jacoco/`, jadi folder itulah yang di-upload sebagai artifact bernama *jacoco-report*. Agar `mvn jacoco:report` menghasilkan laporan, plugin *jacoco-maven-plugin* dengan goal `prepare-agent` harus sudah ada di `pom.xml` (contohnya ada di Bagian E).

BAGIAN D: CODE COVERAGE & INTEGRASI

Soal 7

1. Total statement

10 + 15 + 20 + 5 = **50 statement**.

2. Total statement yang tidak tereksekusi / Miss

0 + 2 + 8 + 1 = **11 statement**. Cek coverage total: (50 - 11) / 50 = 39/50 = 78%, sesuai dengan angka TOTAL pada report.

3. File dengan coverage paling rendah

File **src/payment.py** dengan coverage **60%** (8 dari 20 statement belum dieksekusi).

4. Cara meningkatkan coverage payment.py

- Jalankan `pytest --cov=src --cov-report=term-missing` atau buka laporan HTML (`htmlcov/index.html`) untuk melihat nomor baris yang belum tereksekusi (8 statement yang Miss).
- Analisis baris tersebut: biasanya berada di percabangan `if/else`, penanganan error/exception, atau kondisi khusus yang belum pernah dipicu oleh test.
- Tambahkan test case baru untuk path yang belum tercakup, misalnya pembayaran dengan saldo tidak cukup, nominal nol atau negatif, metode pembayaran tidak valid, dan skenario gagal lainnya (pakai `pytest.raises` untuk exception).
- Jalankan ulang test dan pastikan coverage naik, minimal sampai 80% sesuai target, lalu pasang threshold di pipeline (misalnya `--cov-fail-under=80`) agar coverage tidak turun lagi.

5. Apakah coverage 78% sudah cukup baik?

Menurut saya **belum cukup baik**. Pertama, target coverage yang dipakai di materi adalah 80-90% untuk statement dan line coverage, sedangkan 78% masih di bawah batas minimum 80%. Kedua, angka total ini menutupi masalah di tingkat file: payment.py baru 60%, padahal modul pembayaran umumnya kritis dan risiko bug-nya tinggi. Ketiga, masih ada 11 statement yang belum pernah dijalankan sama sekali, sehingga bug di bagian itu tidak akan terdeteksi. Meskipun begitu, perlu diingat bahwa coverage yang tinggi pun belum otomatis menjamin kualitas, karena coverage hanya mengukur kode yang dieksekusi, bukan apakah assertion-nya sudah benar. Jadi coverage sebaiknya dipakai sebagai indikator, ditambah kualitas test case itu sendiri.

BAGIAN E: STUDI KASUS TERPADU

Soal 8: Project BankAccount

Project dibuat dengan Java 17, Maven, JUnit 5, dan JaCoCo, dengan struktur sebagai berikut:

```
testing-pratikum/
|-- .github/
|   `-- workflows/
|       `-- test.yml
|-- src/
|   |-- main/java/com/stmik/BankAccount.java
|   `-- test/java/com/stmik/BankAccountTest.java
|-- .gitignore
`-- pom.xml
```

1. Kode program: BankAccount.java

```
package com.stmik;

public class BankAccount {

    private double balance;

    public BankAccount() {
        this.balance = 0;
    }

    public void deposit(double amount) {
        if (amount < 0) {
            throw new IllegalArgumentException("Jumlah setoran tidak boleh negatif");
        }
        balance += amount;
    }

    public void withdraw(double amount) {
        if (amount < 0) {
            throw new IllegalArgumentException("Jumlah penarikan tidak boleh negatif");
        }
        if (amount > balance) {
            throw new IllegalStateException("Saldo tidak mencukupi");
        }
        balance -= amount;
    }

    public double getBalance() {
        return balance;
    }
}
```

2. Unit test: BankAccountTest.java (10 test case)

```
package com.stmik;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class BankAccountTest {

    private static final double DELTA = 0.001;
    private BankAccount account;

    @BeforeEach
    public void setUp() {
        account = new BankAccount();
    }

    @Test
    public void testSaldoAwalNol() {
        assertEquals(0.0, account.getBalance(), DELTA);
    }

    @Test
```

```
    public void testDepositMenambahSaldo() {
        account.deposit(100000);
        assertEquals(100000.0, account.getBalance(), DELTA);
    }

    @Test
    public void testDepositBeberapaKali() {
        account.deposit(50000);
        account.deposit(25000);
        assertEquals(75000.0, account.getBalance(), DELTA);
    }

    @Test
    public void testDepositNegatifMelemparException() {
        assertThrows(IllegalArgumentException.class, () -> account.deposit(-1));
    }

    @Test
    public void testWithdrawMengurangiSaldo() {
        account.deposit(100000);
        account.withdraw(40000);
        assertEquals(60000.0, account.getBalance(), DELTA);
    }

    @Test
    public void testWithdrawSebesarSaldo() {
        account.deposit(50000);
        account.withdraw(50000);
        assertEquals(0.0, account.getBalance(), DELTA);
    }

    @Test
    public void testWithdrawNegatifMelemparException() {
        account.deposit(100000);
        assertThrows(IllegalArgumentException.class, () -> account.withdraw(-1));
    }

    @Test
    public void testWithdrawMelebihiSaldoMelemparException() {
        account.deposit(50000);
        IllegalStateException e = assertThrows(IllegalStateException.class,
                () -> account.withdraw(50001));
        assertEquals("Saldo tidak mencukupi", e.getMessage());
    }

    @Test
    public void testSaldoTidakBerubahSetelahWithdrawGagal() {
        account.deposit(50000);
        assertThrows(IllegalStateException.class, () -> account.withdraw(70000));
        assertEquals(50000.0, account.getBalance(), DELTA);
    }

    @Test
    public void testDepositNolTidakMengubahSaldo() {
        account.deposit(0);
        assertEquals(0.0, account.getBalance(), DELTA);
    }
}
```

3. pom.xml (JUnit 5 + JaCoCo)

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
                             http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.stmik</groupId>
  <artifactId>testing-praktikum</artifactId>
  <version>1.0-SNAPSHOT</version>

  <properties>
    <maven.compiler.release>17</maven.compiler.release>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>
```

```
  <dependencies>
    <dependency>
      <groupId>org.junit.jupiter</groupId>
      <artifactId>junit-jupiter</artifactId>
      <version>5.10.2</version>
      <scope>test</scope>
    </dependency>
  </dependencies>

  <build>
    <plugins>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-surefire-plugin</artifactId>
        <version>3.2.5</version>
      </plugin>
      <plugin>
        <groupId>org.jacoco</groupId>
        <artifactId>jacoco-maven-plugin</artifactId>
        <version>0.8.11</version>
        <executions>
          <execution>
            <id>prepare-agent</id>
            <goals><goal>prepare-agent</goal></goals>
          </execution>
          <execution>
            <id>report</id>
            <phase>test</phase>
            <goals><goal>report</goal></goals>
          </execution>
        </executions>
      </plugin>
    </plugins>
  </build>
</project>
```

4. GitHub Actions: .github/workflows/test.yml

```
name: Java CI with Maven

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  test:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Setup JDK 17
        uses: actions/setup-java@v4
        with:
          java-version: '17'
          distribution: 'temurin'

      - name: Run tests
        run: mvn -B test

      - name: Generate coverage report
        run: mvn -B jacoco:report

      - name: Upload coverage report
        uses: actions/upload-artifact@v4
        with:
          name: coverage-report
          path: target/site/jacoco/
```

5. Upload ke GitHub

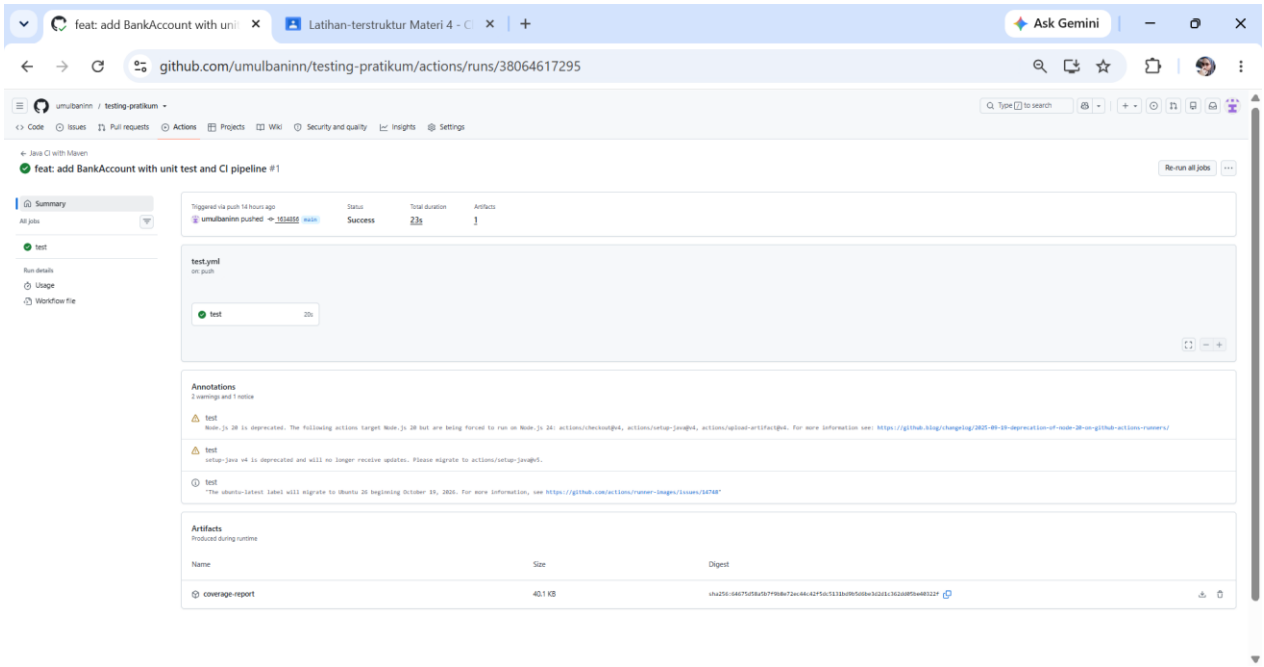
```
git add .
git commit -m "feat: add BankAccount with unit test and CI pipeline"
git push origin main
```

Setelah push, pipeline dicek di tab *Actions* pada repository. Hasil artifact *coverage-report* bisa diunduh dari halaman run workflow tersebut.

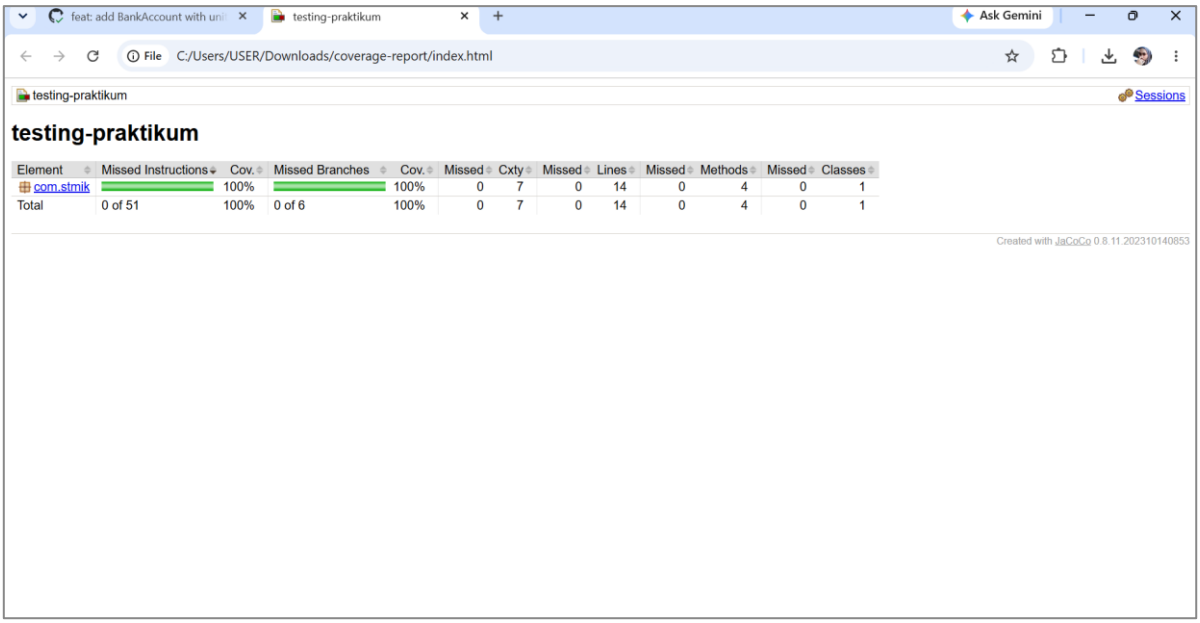
Output Bagian E

Output	Keterangan
Link repository GitHub	https://github.com/umulbaninn/testing-pratikum
Screenshot pipeline sukses	Lihat Gambar E.1 di bawah
Screenshot coverage report	Lihat Gambar E.2 di bawah

Bukti pipeline berjalan sukses dan hasil coverage dari project BankAccount ditampilkan pada gambar berikut.



Gambar E.1. Hasil run workflow di GitHub Actions: status Success



Gambar E.2. Laporan coverage JaCoCo (index.html)

Hasil coverage: instruction coverage 100% (0 dari 51 terlewat) dan branch coverage 100% (0 dari 6 terlewat), di atas target 80% pada materi, karena seluruh cabang pada method deposit dan withdraw sudah diuji.

LINK REPOSITORY GITHUB

Keterangan	**Link**
Repository latihan Git (Soal 2)	https://github.com/umulbaninn/latihan-git-1224023
Repository studi kasus (Soal 8)	https://github.com/umulbaninn/testing-pratikum
Folder pengumpulan tugas	https://github.com/umulbaninn/testing-pratikum/tree/main/Tugas/Tugas04